

Implementación escalar y vectorial con AVX2

Nombre del autor

Resumen—Este proyecto implementa un normalizador estadístico y extractor de descriptores (suma, media, varianza, mínimo, máximo y *z-score*) para arreglos `float32` en x86-64 mediante NASM, enlazado con un controlador de evaluación escrito en C. Se compara una implementación escalar, basada en instrucciones SSE, con una implementación vectorial optimizada mediante AVX2 y registros YMM de 256 bits. La evaluación cuantitativa demuestra aceleraciones de hasta $5,80\times$ y reducciones del número de instrucciones de hasta $7,19\times$ cuando los datos residen en los niveles de caché L1, L2 o L3. Al trabajar desde DRAM, la aceleración converge a $2,44\times$ debido a la saturación del ancho de banda de memoria, fenómeno conocido como *Memory Wall*.

Index Terms—SIMD, AVX2, x86-64, NASM, System V AMD64 ABI, normalización *z-score*, speedup, IPC, *Memory Wall*.

I. INTRODUCCIÓN Y OBJETIVOS

El cálculo de estadísticos descriptivos y la normalización tipificada, definida para cada elemento como

$$z_i = \frac{x_i - \mu}{\sigma}, \quad (1)$$

constituyen operaciones centrales en el procesamiento de señales, la visión artificial y el aprendizaje profundo. Ante el agotamiento del escalado de Dennard y los límites térmicos que restringen el aumento de la frecuencia, el incremento del rendimiento microarquitectónico depende críticamente del paralelismo a nivel de datos (DLP, *Data-Level Parallelism*). La vectorización manual mediante extensiones SIMD, en particular AVX2, permite procesar múltiples carriles simultáneamente con cada instrucción, reduciendo la latencia computacional y la carga sobre el decodificador frente al modelo secuencial escalar clásico (SISD).

I-A. Objetivo general

Diseñar, implementar en ensamblador x86-64 mediante NASM y evaluar cuantitativamente dos núcleos de cómputo funcionalmente equivalentes: uno escalar, basado en SSE, y otro vectorial, basado en AVX2. Ambos se integran con un controlador de pruebas en C bajo la convención System V AMD64 ABI para el cálculo de descriptores estadísticos y la normalización *z-score* de arreglos `float32`. Además, se analiza su rendimiento, aceleración e IPC en relación con la jerarquía de memoria.

I-B. Objetivos específicos

- OA1. Explicar y contrastar el modelo de ejecución SIMD frente a SISD, analizando el procesamiento simultáneo de ocho elementos por registro YMM frente al procesamiento escalar elemento a elemento.
- OA2. Describir la evolución de las extensiones vectoriales x86 (MMX, SSE, AVX, AVX2 y AVX-512) y fundamentar

la selección de AVX2 como un compromiso adecuado entre compatibilidad y rendimiento.

- OA3. Desarrollar núcleos de cómputo en NASM para x86-64 mediante instrucciones empaquetadas, entre ellas `vmovaps`, `vaddps`, `vsubps`, `vmulps`, `vdivps`, `vminps` y `vmaxps`, cumpliendo la convención System V AMD64 ABI.
- OA4. Aplicar alineación estricta de memoria a 32 bytes mediante `aligned_alloc` y resolver el procesamiento del remanente mediante el enmascaramiento bit a bit `n & ~7`, seguido de un bucle escalar de cierre (*tail loop*).
- OA5. Emplear GDB para inspeccionar registros vectoriales con `print $ymm0.v8_float`, punteros y memoria con `x/8fw`, a fin de validar la corrección matemática y la ausencia de excepciones de protección general (#GP).
- OA6. Medir experimentalmente la latencia mediante `clock_gettime`, así como el speedup y el IPC mediante `perf stat`, para tamaños $N \in [10^3, 2 \times 10^7]$. Las variaciones del factor de aceleración se correlacionan con la jerarquía de cachés y el cuello de botella de DRAM (*Memory Wall*).
- OA7. Argumentar las decisiones de diseño de bajo nivel, incluido el árbol de reducción horizontal con `vextractf128` y la mitigación de las penalizaciones de transición AVX–SSE mediante `vzeroupper`.

I-C. Comparación conceptual entre SIMD y SISD

La Tabla I resume las diferencias principales entre el procesamiento escalar y el procesamiento vectorial empleado en este trabajo.

I-D. Factor de aceleración teórico

En una arquitectura SIMD que ejecuta operaciones homogéneas sobre datos contiguos, la aceleración ideal está determinada por el cociente entre el ancho del registro vectorial, W_{vector} , y el tamaño del dato escalar, W_{dato} :

$$S_{\text{teórico}} = \frac{T_{\text{SISD}}}{T_{\text{SIMD}}} = \frac{W_{\text{vector}}}{W_{\text{dato}}} = \frac{256 \text{ bits}}{32 \text{ bits}} = 8,0 \times . \quad (2)$$

En este caso, $W_{\text{vector}} = 256$ bits corresponde al ancho de un registro YMM de AVX2, mientras que $W_{\text{dato}} = 32$ bits corresponde al formato `float32` de IEEE 754. Por tanto, una sola instrucción empaquetada AVX2 puede realizar el equivalente aritmético de ocho instrucciones escalares independientes, de modo que idealmente

$$\text{Throughput}_{\text{SIMD}} = 8 \text{ Throughput}_{\text{SISD}}. \quad (3)$$

Este límite supone ausencia de dependencias, sobrecostes de reducción y restricciones de memoria; por ello, el speedup observado en la práctica puede ser menor.

Cuadro I: Comparación conceptual entre los modelos escalar SISD y vectorial SIMD.

Criterio	Modelo escalar (SISD)	Modelo vectorial AVX2 (SIMD)
Formato del dato	Escalar simple: un elemento <code>float32</code>	Empaquetado: ocho elementos <code>float32</code>
Registro utilizado	<code>xmm0–xmm7</code> , usando el carril bajo de 32 bits	<code>ymm0–ymm5</code> , usando los 256 bits
Operaciones por instrucción	Una operación de punto flotante	Ocho operaciones simultáneas por instrucción
Sobrecarga de control	N iteraciones con saltos y ramas	$\lfloor N/8 \rfloor$ iteraciones vectoriales más el remanente escalar
Presión sobre el decodificador	Alta, debido a la mayor emisión de microoperaciones	Reducida; se observaron hasta $7,19\times$ menos instrucciones
Reducción de sumas	Inmediata mediante un acumulador secuencial	Árbol horizontal mediante <code>vextractf128</code> y permutaciones
Alineación de memoria	Estándar, sin requisitos críticos para las cargas escalares	Alineación de 32 bytes para <code>vmovaps</code> , necesaria para evitar excepciones <code>#GP</code>

II. ENTORNO DE PRUEBAS

II-A. Hardware y soporte de AVX2

La evaluación experimental se realizó sobre un procesador Intel Core i3-1215U de 12.^a generación, basado en la arquitectura Alder Lake y fabricado mediante el nodo Intel 7. Este procesador presenta una microarquitectura híbrida asimétrica compuesta por dos núcleos de rendimiento (P-cores Golden Cove con soporte para *Hyper-Threading*) y cuatro núcleos de eficiencia (E-cores Gracemont sin multihilo), para un total de seis núcleos físicos y ocho hilos lógicos.

La jerarquía de memoria dispone de cachés privadas L1 de datos (L1d) de 48 KiB por P-core y 32 KiB por E-core. Cada P-core cuenta con una caché L2 privada de 1,25 MiB, mientras que el clúster de E-cores comparte una caché L2 de 2 MiB. El último nivel corresponde a una caché L3 unificada de 10 MiB, denominada Intel Smart Cache, con líneas de caché de 64 bytes.

La inspección mediante `lscpu` y el análisis de `/proc/cpuinfo` confirmaron el soporte nativo de AVX2 sobre registros YMM de 256 bits, además de las extensiones FMA3 y SSE4.2. Las características principales del sistema se resumen en la Tabla II.

Cuadro II: Características del entorno de hardware.

Característica	Valor
Modelo de CPU	Intel Core i3-1215U (12. ^a generación, Alder Lake, Intel 7)
Arquitectura	x86-64 híbrida: 2 P-cores Golden Cove y 4 E-cores Gracemont
Núcleos/hilos	6 núcleos físicos (2P + 4E) y 8 hilos lógicos
Frecuencia base/turbo	P-cores: 1,20 GHz/ 4,40 GHz; E-cores: 0,90 GHz/ 3,30 GHz
Caché L1d	48 KiB/P-core (12 vías), 32 KiB/E-core (8 vías); total: 224 KiB
Caché L1i	32 KiB/P-core (8 vías), 64 KiB/E-core (8 vías); total: 320 KiB
Caché L2	1,25 MiB/P-core y 2 MiB compartidos por el clúster de E-cores; total: 4,5 MiB
Caché L3 (LLC)	10 MiB unificada y compartida (Intel Smart Cache)
Línea de caché	64 bytes
Flags relevantes	<code>avx</code> , <code>avx2</code> , <code>fma</code> , <code>sse4_1</code> , <code>sse4_2</code> , <code>bmi1</code> , <code>bmi2</code> , <code>popcnt</code>

II-B. Herramientas utilizadas

El desarrollo, la instrumentación y el análisis de los núcleos de cómputo se llevaron a cabo mediante el conjunto de herramientas nativo de GNU/Linux x86-64. El flujo de trabajo integró el ensamblado de bajo nivel con metadatos DWARF, la

compilación modular del arnés de pruebas conforme a la convención System V AMD64 ABI, la inspección interactiva de registros vectoriales y la caracterización microarquitectónica no invasiva mediante la unidad de monitoreo de rendimiento de la CPU (PMU, *Performance Monitoring Unit*). La Tabla III detalla las versiones y el propósito de cada herramienta.

III. IMPLEMENTACIÓN ESCALAR

III-A. Descripción del algoritmo

El procesamiento estadístico escalar se divide en dos rutinas desacopladas, `compute_stats` y `normalize_array`, implementadas íntegramente en ensamblador x86-64. La primera calcula los descriptores estadísticos mediante dos recorridos del arreglo, mientras que la segunda produce el arreglo normalizado a partir de la media y la desviación estándar obtenidas.

III-A1. Algoritmo de dos pasadas en `compute_stats`:

En la primera pasada se fusionan la reducción de suma y el seguimiento de los extremos. Los tres acumuladores se inician con el primer elemento:

$$S = x_0, \quad x_{\min} = x_0, \quad x_{\max} = x_0. \quad (4)$$

Luego se recorre el arreglo desde $i = 1$ hasta $N - 1$, actualizando simultáneamente

$$S = \sum_{i=0}^{N-1} x_i, \quad x_{\min} = \min_i(x_i), \quad x_{\max} = \max_i(x_i). \quad (5)$$

Al finalizar, N se convierte a punto flotante y se calcula la media aritmética:

$$\mu = \frac{S}{N} = \frac{1}{N} \sum_{i=0}^{N-1} x_i. \quad (6)$$

La fusión de estas tres operaciones aprovecha las líneas de caché que ya se encuentran en L1d y evita recorridos adicionales para obtener el mínimo y el máximo.

La segunda pasada calcula la varianza poblacional. Con la media retenida en un registro XMM se acumula la suma de desviaciones cuadráticas

$$SS = \sum_{i=0}^{N-1} (x_i - \mu)^2, \quad (7)$$

y posteriormente se divide entre N :

$$\sigma^2 = \frac{SS}{N} = \frac{1}{N} \sum_{i=0}^{N-1} (x_i - \mu)^2. \quad (8)$$

Cuadro III: Herramientas utilizadas en el entorno de pruebas.

Herramienta	Versión	Propósito en el proyecto
NASM	3.01 (compatible con 2.16+)	Ensamblado de las rutinas escalares y vectoriales (stats_scalar.asm y stats_vector.asm) en objetos ELF64 con símbolos de depuración DWARF mediante <code>-f elf64 -g -F dwarf</code> .
GCC	15.2.0	Compilación del controlador <code>driver.c</code> con <code>-O2 -std=gnu11</code> , gestión de búferes alineados a 32 bytes mediante <code>aligned_alloc</code> y enlace final del ejecutable.
GDB	17.1	Depuración de bajo nivel, validación de la alineación de memoria mediante <code>addr% 32 == 0</code> e inspección de los ocho carriles flotantes de registros YMM con <code>\$ymm0.v8_float</code> .
perf	6.13	Perfilado mediante los contadores de hardware de la CPU: ciclos de reloj, instrucciones retiradas, IPC, fallos de caché L1d y fallos en la LLC.
GNU Make	4.4.1	Automatización del ciclo de compilación, construcción de los binarios <code>norm_scalar</code> y <code>norm_vector</code> , ejecución de las suites de prueba y realización sistemática de benchmarks.

Se descartó deliberadamente la expresión de una sola pasada

$$\sigma^2 = \frac{1}{N} \sum_{i=0}^{N-1} x_i^2 - \mu^2, \quad (9)$$

porque puede sufrir cancelación catastrófica al restar magnitudes grandes y muy próximas. En precisión simple IEEE 754, cuya mantisa efectiva es de 24 bits, esta pérdida de significancia puede amplificar el error de redondeo e incluso producir varianzas negativas espurias.

III-A2. Normalización en `normalize_array`: La rutina recorre el arreglo una vez y aplica la transformación

$$z_i = \frac{x_i - \mu}{\sigma}. \quad (10)$$

Cuando $\sigma = 0,0$, todos los elementos presentan dispersión nula. En este caso, la rutina copia cada x_i a la salida sin efectuar la división, con lo que evita resultados indeterminados, valores NaN o infinitos.

III-A3. Complejidad algorítmica: Ambas rutinas presentan complejidad temporal $O(N)$. En particular, `compute_stats` realiza dos recorridos, equivalentes a $2N$ iteraciones, y `normalize_array` realiza un recorrido adicional de N iteraciones. La complejidad espacial auxiliar es $O(1)$, pues el cálculo no reserva memoria dinámica y mantiene sus acumuladores en registros de la CPU.

III-B. Decisiones de implementación

III-B1. Uso de registros escalares XMM: La implementación emplea exclusivamente el carril inferior de 32 bits de los registros `xmm0`–`xmm6`. Las instrucciones con sufijo `ss` (*Scalar Single-Precision*), como `addss`, `subss`, `mulss` y `divss`, procesan un elemento `float32` por instrucción y corresponden al modelo SISD utilizado como línea base.

III-B2. Convención System V AMD64 ABI: En `compute_stats`, los parámetros se reciben en el orden establecido por la ABI: `rdi` contiene `arr`, `rsi` contiene N y `rdx`, `rcx`, `r8` y `r9` contienen los punteros de salida para la media, la varianza, el mínimo y el máximo, respectivamente. En `normalize_array`, `rdi`, `rsi` y `edx` contienen los punteros de entrada y salida y la longitud; los argumentos flotantes μ y σ se reciben en `xmm0` y `xmm1`.

Para conservar los punteros durante las dos pasadas de `compute_stats`, se trasladan a `r12`–`r15`, `rbx` y `rbp`. Estos registros no volátiles (*callee-saved*) se guardan mediante `push` en el prólogo y se restauran con `pop`, en orden inverso,

antes del retorno. De esta forma se preserva el estado esperado por la función invocadora escrita en C.

III-B3. Selección de extremos sin bifurcaciones: Las instrucciones `minss` y `maxss` actualizan los extremos directamente en hardware, sin recurrir a `comiss` ni a saltos condicionales como `ja` o `jb`. Este enfoque *branchless* evita penalizaciones por predicciones de salto incorrectas, que en procesadores modernos pueden costar del orden de decenas de ciclos.

III-B4. Detección de varianza nula: Antes de normalizar, `normalize_array` construye la constante `0,0` mediante `xorps xmm2, xmm2` y la compara con la desviación estándar usando `ucomiss xmm1, xmm2`. Si $\sigma = 0,0$, el control se desvía a `.copy_loop`; así se evita la división por cero y la generación de NaN o $\pm\infty$.

III-C. Fragmentos de código comentados

```

1 ; PASADA 1: suma, minimo y maximo
2 movss  xmm0, [r12]          ; suma = arr[0]
3 movaps  xmm1, xmm0          ; min = arr[0]
4 movaps  xmm2, xmm0          ; max = arr[0]
5 mov     eax, 1               ; i = 1
6 .pass1_loop:
7     cmp     eax, r13d         ; i >= n?
8     jge     .pass1_done
9     movss  xmm3, [r12 + rax*4] ; arr[i]
10    addss  xmm0, xmm3         ; suma += arr[i]
11    minss  xmm1, xmm3         ; min = min(min, arr[i])
12    maxss  xmm2, xmm3         ; max = max(max, arr[i])
13    inc     eax
14    jmp     .pass1_loop
15 .pass1_done:
16    cvtsi2ss xmm4, r13d       ; xmm4 = (float)n
17    movaps  xmm5, xmm0
18    divss  xmm5, xmm4         ; mean = suma / n
19    movss  [rbx], xmm1        ; *min = min
20    movss  [rbp], xmm2        ; *max = max
21    movss  [r14], xmm5        ; *mean = mean
22
23 ; PASADA 2: varianza poblacional
24 xor     eax, eax
25 xorps   xmm6, xmm6          ; acumulador = 0.0
26 .pass2_loop:
27     cmp     eax, r13d
28     jge     .pass2_done
29     movss  xmm3, [r12 + rax*4]
30     subss  xmm3, xmm5         ; arr[i] - mean
31     mulss  xmm3, xmm3         ; (arr[i] - mean)^2
32     addss  xmm6, xmm3
33     inc     eax
34     jmp     .pass2_loop
35 .pass2_done:
36     divss  xmm6, xmm4         ; var = SS / n
37     movss  [r15], xmm6       ; *var = var

```

```

38
39 ; NORMALIZACION: z-score o copia si sigma = 0
40 ; rdi=in, rsi=out, edx=n, xmm0=mean, xmm1=stddev
41 test     edx, edx
42 jle      .norm_done
43 xorps    xmm2, xmm2
44 ucomiss  xmm1, xmm2
45 je       .copy_loop
46 xor      eax, eax
47 .norm_loop:
48     cmp    eax, edx
49     jge     .norm_done
50     movss  xmm3, [rdi + rax*4]
51     subss  xmm3, xmm0
52     divss  xmm3, xmm1
53     movss  [rsi + rax*4], xmm3
54     inc    eax
55     jmp    .norm_loop
56 .copy_loop:
57     xor     eax, eax
58 .copy_inner:
59     cmp    eax, edx
60     jge     .norm_done
61     movss  xmm3, [rdi + rax*4]
62     movss  [rsi + rax*4], xmm3
63     inc    eax
64     jmp    .copy_inner
65 .norm_done:
66     ret

```

Listing 1: Bucles principales de stats_scalar.asm.

La Tabla IV resume la asignación de registros en las dos rutinas. Para conservarla dentro de una sola columna se utiliza una tipografía compacta y descripciones abreviadas.

Cuadro IV: Asignación de registros en la implementación escalar.

Registro	Tipo	compute_stats	normalize_array
r12/rdi	Entero 64 b	Base de arr	Base de in
rsi	Entero 64 b	No usado	Base de out
r13d/edx	Entero 32 b	Longitud N	Longitud N
eax/rax	Entero 32/64 b	Índice i	Índice i
xmm0	Escalar 32 b	Suma $\sum x_i$	Media μ
xmm1	Escalar 32 b	Mínimo	Desviación σ
xmm2	Escalar 32 b	Máximo	Constante 0,0
xmm3	Escalar 32 b	Operando temporal	Operando temporal
xmm4	Escalar 32 b	N convertido a float	No usado
xmm5	Escalar 32 b	Media μ	No usado
xmm6	Escalar 32 b	SS y varianza σ^2	No usado
r14, r15, rbx, rbp	Punteros 64 b	Destinos de media, varianza, mínimo y máximo	No usados

La evidencia de inspección en bajo nivel de estos registros y la alineación de memoria se detallan en la Sección VII (Figuras 8 a 13).

IV. IMPLEMENTACIÓN VECTORIAL CON AVX2

IV-A. Elección de AVX2

El conjunto de instrucciones AVX2 (*Advanced Vector Extensions* 2) se seleccionó después de evaluar la capacidad, disponibilidad y coste operativo de las principales extensiones SIMD de x86-64.

Frente a SSE, que utiliza registros XMM de 128 bits y procesa cuatro valores float32 por instrucción, AVX2 duplica el ancho a 256 bits y permite operar sobre ocho valores simultáneamente. En consecuencia, el techo ideal asociado únicamente al ancho vectorial aumenta de $4,0\times$ a $8,0\times$.

Además, la codificación tradicional de SSE emplea instrucciones destructivas de dos operandos, por ejemplo addps dest, src, mientras que el prefijo VEX de AVX2 permite instrucciones no destructivas de tres operandos:

$$\begin{aligned} & \text{vaddps dest, src1, src2} \\ \Rightarrow & \text{dest} = \text{src1} + \text{src2}. \end{aligned} \quad (11)$$

Esto preserva las fuentes y reduce la necesidad de copias auxiliares con movaps.

AVX-512 elevaría el ancho a 512 bits, equivalente a 16 valores float32. Sin embargo, no está disponible de forma homogénea en el Intel Core i3-1215U utilizado: sus E-cores Gracemont no incorporan unidades AVX-512 y la plataforma mantiene deshabilitada esta extensión para ofrecer un repertorio uniforme entre núcleos. En las plataformas que sí la implementan, las cargas AVX-512 intensivas también pueden provocar reducciones de frecuencia dependientes del procesador y del presupuesto térmico. A esto se añade la fragmentación de AVX-512 en subconjuntos como F, CD, BW, DQ y VL. AVX2 ofrece, por tanto, una cobertura más amplia y predecible en equipos x86-64 modernos.

La Tabla V resume los criterios de selección. Se mantiene dentro de una columna mediante descripciones compactas.

Cuadro V: Comparación de extensiones vectoriales x86.

Criterio	SSE	AVX2	AVX-512
Registro	128 b XMM	256 b YMM	512 b ZMM
float32/vector	4	8	16
Operandos	2, destructivos	3, VEX	3, EVEX y máscara
Techo ideal	$4\times$	$8\times$	$16\times$
Compatibilidad	Muy amplia	Amplia	Restringida en consumo
Efecto térmico	Bajo	Bajo o moderado	Puede ser alto

En síntesis, AVX2 constituye el punto de equilibrio para este trabajo: ofrece ocho operaciones simultáneas por instrucción, está soportado por todos los núcleos del sistema experimental y evita depender de una extensión ausente en la plataforma objetivo.

IV-B. Organización del procesamiento vectorial

Los datos se agrupan en bloques de ocho números IEEE 754 de precisión simple. Cada registro YMM se interpreta como ocho carriles independientes:

$$\text{YMM} = [f_0, f_1, f_2, f_3 \mid f_4, f_5, f_6, f_7]. \quad (12)$$

Los acumuladores principales ocupan ymm0–ymm5. Cada iteración consume $8 \times 4 = 32$ bytes contiguos; el índice se conserva en eax/rax y avanza mediante add eax, 8. El modo de direccionamiento SIB convierte el índice de elemento en desplazamiento de bytes con expresiones como [rdi + rax*4] o [r12 + rax*4].

Las principales instrucciones VEX utilizadas son las siguientes:

- **vmovaps**: carga o almacena ocho valores empaquetados. Las direcciones usadas por esta implementación se mantienen alineadas a 32 bytes, condición necesaria para esta variante alineada.

- `vaddps`, `vsubps`, `vmulps` y `vdivps`: ejecutan suma, resta, multiplicación y división en los ocho carriles.
- `vminps` y `vmaxps`: actualizan en paralelo los ocho mínimos o máximos parciales.
- `vbroadcastss`: replica el valor escalar del carril inferior de un XMM en los ocho carriles de un YMM, es decir, $[s, s, s, s, s, s, s, s]$.

La evidencia de inspección en bajo nivel de estos registros y la alineación de memoria se detallan en la Sección VII (Figuras 8 a 13).

IV-C. Reducción horizontal

Al concluir el bucle vectorial, cada acumulador contiene ocho resultados parciales:

$$\text{YMM} = [c_0, c_1, c_2, c_3 \mid c_4, c_5, c_6, c_7]. \quad (13)$$

Para producir un único resultado escalar se aplican árboles de reducción de profundidad logarítmica.

IV-C1. Reducción de suma: Primero, `vextractf128` extrae los carriles 4–7. La instrucción `vaddps` combina ambas mitades y deja cuatro sumas parciales:

$$[c_0 + c_4, c_1 + c_5, c_2 + c_6, c_3 + c_7]. \quad (14)$$

Dos aplicaciones de `vhaddps` colapsan primero los pares adyacentes y luego las dos sumas restantes, por lo que

$$\text{xmm0}[0] = \sum_{i=0}^7 c_i. \quad (15)$$

IV-C2. Reducción de mínimo y máximo: x86-64 no proporciona instrucciones `vhminps` o `vhmaxps`. La reducción se construye mediante `vextractf128`, operaciones de mínimo o máximo y dos permutaciones. La máscara `0x4E` reorganiza los carriles como $[2, 3, 0, 1]$, mientras que `0xB1` los reorganiza como $[1, 0, 3, 2]$. El siguiente fragmento muestra la reducción del mínimo:

```
1 vextractf128 xmm3, ymm1, 1
2 vminps      xmm1, xmm1, xmm3
3 vshufps     xmm3, xmm1, xmm1, 0x4E
4 vminps      xmm1, xmm1, xmm3
5 vshufps     xmm3, xmm1, xmm1, 0xB1
6 vminps      xmm1, xmm1, xmm3
7 ; xmm1[0] contiene el minimo global
```

Listing 2: Árbol horizontal para reducir el mínimo.

La reducción del máximo repite el mismo árbol sobre `ymm2/xmm2`, sustituyendo `vminps` por `vmaxps`.

IV-D. Manejo del remanente

Cuando N no es múltiplo de ocho, una carga vectorial final accedería a posiciones ajenas al arreglo y podría atravesar un límite de página. Para evitarlo, se calcula primero el número de elementos que forman bloques completos:

$$N_{\text{vec}} = N \& (\sim 7) = N - (N \bmod 8). \quad (16)$$

La máscara de 32 bits es

$$\sim 7 = 0xFFFFFFFF8, \quad (17)$$

de modo que `and ecx, ~7` apaga los tres bits menos significativos y redondea N hacia abajo al múltiplo de ocho más cercano.

Si $N < 8$, entonces $N_{\text{vec}} = 0$ y el bucle AVX2 se omite por completo. Para $N \bmod 8 \in \{1, \dots, 7\}$, un bucle escalar de cierre comienza en $i = N_{\text{vec}}$ y procesa los elementos restantes con instrucciones VEX escalares: `vmovss`, `vaddss`, `vminss`, `vmaxss`, `vsubss`, `vmulss` y `vdivss`. Cada `vmovss` lee exactamente cuatro bytes y los resultados se acumulan sobre los registros XMM que ya contienen la reducción horizontal.

IV-E. Fragmentos de código comentados

Los fragmentos siguientes corresponden a los núcleos de `asm/vector/stats_vector.asm`.

```
1 .pass1_vec_loop:
2   cmp     eax, ecx
3   jge     .pass1_vec_reduce
4   vmovaps ymm3, [r12 + rax*4]
5   vaddps  ymm0, ymm0, ymm3
6   vminps  ymm1, ymm1, ymm3
7   vmaxps  ymm2, ymm2, ymm3
8   add     eax, 8
9   jmp     .pass1_vec_loop
10
11 .pass1_vec_reduce:
12   ; Suma: ymm0 -> xmm0[0]
13   vextractf128 xmm3, ymm0, 1
14   vaddps  xmm0, xmm0, xmm3
15   vhaddps xmm0, xmm0, xmm0
16   vhaddps xmm0, xmm0, xmm0
17
18   ; Mínimo: ymm1 -> xmm1[0]
19   vextractf128 xmm3, ymm1, 1
20   vminps  xmm1, xmm1, xmm3
21   vshufps xmm3, xmm1, xmm1, 0x4E
22   vminps  xmm1, xmm1, xmm3
23   vshufps xmm3, xmm1, xmm1, 0xB1
24   vminps  xmm1, xmm1, xmm3
```

Listing 3: Primera pasada vectorial y reducción de suma y mínimo.

```
1 .pass1_tail_loop:
2   cmp     eax, r13d
3   jge     .pass1_done
4   vmovss  xmm3, [r12 + rax*4]
5   vaddss  xmm0, xmm0, xmm3
6   vminss  xmm1, xmm1, xmm3
7   vmaxss  xmm2, xmm2, xmm3
8   inc     eax
9   jmp     .pass1_tail_loop
```

Listing 4: Bucle escalar de cierre.

```
1 normalize_array:
2   test    edx, edx
3   jle     .norm_vec_ret
4   vxorps  xmm2, xmm2, xmm2
5   vucomiss xmm1, xmm2
6   je      .norm_copy_loop
7
8   vbroadcastss ymm4, xmm0 ; media
9   vbroadcastss ymm5, xmm1 ; desviacion
10  mov     ecx, edx
11  and     ecx, ~7
12  xor     eax, eax
13  .norm_vec_loop:
14  cmp     eax, ecx
```

```

15     jge     .norm_scalar_tail
16     vmovaps ymm0, [rdi + rax*4]
17     vsubps  ymm0, ymm0, ymm4
18     vdivps  ymm0, ymm0, ymm5
19     vmovaps [rsi + rax*4], ymm0
20     add     eax, 8
21     jmp     .norm_vec_loop

```

Listing 5: Normalización vectorial con difusión de escalares.

IV-E1. Uso de vzeroupper: Después de ejecutar instrucciones AVX de 256 bits, los bits superiores de los registros YMM pueden conservar datos activos. En microarquitecturas sensibles a la transición entre AVX y SSE heredado, retornar directamente a código que usa instrucciones SSE sin prefijo VEX puede introducir una penalización. Por ello, todos los caminos de salida ejecutan

```

1 .norm_vec_done:
2     vzeroupper
3     ret

```

Listing 6: Limpieza del estado superior de los registros YMM.

La instrucción marca como limpios los carriles superiores de los registros YMM y evita depender del comportamiento concreto del código C o de las bibliotecas invocadas después del retorno.

IV-F. Asignación de registros vectoriales

Todos los registros XMM/YMM son volátiles (*caller-saved*) bajo System V AMD64 ABI. Por ello, las rutinas pueden utilizarlos sin guardarlos previamente en la pila. Para conservar las tablas a media columna, la asignación se divide por función.

Cuadro VI: Registros de suma y cálculo estadístico AVX2.

Registro	sum_array	compute_stats
YMM0/XMM0	Suma vectorial y retorno escalar	P1: suma; P2: suma de desviaciones cuadráticas
YMM1/XMM1	Carga vectorial o elemento del remanente	P1: mínimo; P2: carga y $(x - \mu)^2$
YMM2/XMM2	Mitad alta durante la reducción	P1: máximo
YMM3/XMM3	No usado	Carga vectorial, auxiliar de permutación y difusión de μ
YMM4/XMM4	No usado	Divisor N convertido a punto flotante
YMM5/XMM5	No usado	Media escalar $\mu = S/N$

Cuadro VII: Registros de la normalización AVX2.

Registro	Rol en normalize_array
YMM0/XMM0	Carga, resta, división y almacenamiento de ocho valores
YMM1/XMM1	Argumento escalar σ
YMM2/XMM2	Comparación $\sigma = 0,0$ y temporal del remanente
YMM3/XMM3	Temporal escalar del remanente
YMM4/XMM4	Difusión $[\mu, \dots, \mu]$
YMM5/XMM5	Difusión $[\sigma, \dots, \sigma]$

V. CASOS DE PRUEBA

V-A. Comandos de reproducibilidad y ejecución

La compilación desde cero y la verificación funcional automatizada se realizan desde el directorio raíz del proyecto mediante los siguientes comandos:

```

1 # Limpiar y compilar las dos versiones
2 make clean && make
3
4 # Ejecutar la suite funcional completa
5 python3 tools/run_all_tests.py

```

Listing 7: Compilación y ejecución de la suite completa.

Para reproducir manualmente el caso con $N = 15$, que combina un bloque AVX2 de ocho elementos con siete elementos de remanente, se ejecutan ambos binarios en modo detallado y se comparan sus resúmenes estadísticos:

```

1 # Kernel escalar, modo verbose
2 ./bin/norm_scalar data/test_suite/tc-05_in.dat data/
  test_suite/tc-05_sc.dat 1
3
4 # Kernel vectorial AVX2, modo verbose
5 ./bin/norm_vector data/test_suite/tc-05_in.dat data/
  test_suite/tc-05_vc.dat 1
6
7 # Comparar los resúmenes estadísticos
8 cat data/test_suite/tc-05_sc.dat.stats.txt
9 cat data/test_suite/tc-05_vc.dat.stats.txt

```

Listing 8: Ejecución manual del caso con $N = 15$.

V-B. Criterio de validación y tolerancia numérica

La equivalencia entre las implementaciones en ensamblador y el oráculo de referencia escrito en Python se acepta cuando

$$\text{Err}_{\text{rel}} \leq 1,0 \times 10^{-4}. \quad (18)$$

Para un valor producido por ensamblador, v_{asm} , y su referencia, v_{ref} , la discrepancia relativa se calcula como

$$\text{Err}_{\text{rel}} = \frac{|v_{\text{asm}} - v_{\text{ref}}|}{|v_{\text{ref}}| + 10^{-12}}. \quad (19)$$

El término 10^{-12} estabiliza el denominador cuando la referencia es nula o muy cercana a cero.

V-B1. No asociatividad en IEEE 754: La suma en punto flotante `binary32` no satisface, en general, la propiedad asociativa:

$$(a + b) + c \neq a + (b + c). \quad (20)$$

El núcleo escalar acumula los elementos secuencialmente,

$$S_{\text{sc}} = (((x_0 + x_1) + x_2) + \dots + x_{N-1}), \quad (21)$$

mientras que AVX2 mantiene ocho sumas parciales y las combina mediante un árbol horizontal:

$$S_{\text{vec}} = \sum_{j=0}^7 \left(\sum_{k=0}^{\lfloor N/8 \rfloor - 1} x_{8k+j} \right) + S_{\text{tail}}. \quad (22)$$

Cada operación intermedia redondea el resultado a la precisión representable. Por ello, la reasociación inherente al procesamiento SIMD puede producir diferencias del orden de 10^{-7} a 10^{-6} , sin que estas impliquen un error algorítmico.

V-C. Resultados de los casos de prueba

La Tabla VIII resume los seis casos borde obligatorios. Debido al número de resultados comparados, esta tabla utiliza excepcionalmente el ancho de las dos columnas.

Cuadro VIII: Resultados de los casos de prueba.

Caso	Entrada	Referencia	Escalar	AVX2	Resultado
1	Arreglo vacío, $N = 0$; archivo binario sin elementos	Retorno seguro: $\mu = 0,0$, $\sigma^2 = 0,0$	$\mu = 0,0$, $\sigma^2 = 0,0$	$\mu = 0,0$, $\sigma^2 = 0,0$	Pasa
2	Elemento único, $N = 1$; $x_0 = 42,0$	$\mu = 42,0$, $\sigma^2 = 0,0$; salida $[42,0]$	$\mu = 42,0$, $\sigma^2 = 0,0$; salida $[42,0]$	$\mu = 42,0$, $\sigma^2 = 0,0$; salida $[42,0]$	Pasa
3	Remanente puro, $N = 7 < 8$; valores en $[-100, 100]$	$\mu = -0,6737$, $\sigma^2 = 1547,18$	$\mu = -0,6737$, $\sigma^2 = 1547,18$	$\mu = -0,6737$, $\sigma^2 = 1547,18$	Pasa
4	Vector y remanente, $N = 15$; un bloque AVX2 y siete elementos en el <i>tail</i>	$\mu = 19,4100$, $\sigma^2 = 2381,27$	$\mu = 19,4100$, $\sigma^2 = 2381,27$	$\mu = 19,4100$, $\sigma^2 = 2381,27$	Pasa
5	Varianza nula, $N = 1000$; $x_i = 5,0$ para todo i	$\mu = 5,0$, $\sigma^2 = 0,0$; salida clonada	$\mu = 5,0$, $\sigma^2 = 0,0$; salida clonada	$\mu = 5,0$, $\sigma^2 = 0,0$; salida clonada	Pasa
6	Valores extremos, $N = 70$; rango $[-10^6, 10^6]$ con valores del orden de $\pm 10^{-4}$	$\mu = 0,0$, $\sigma^2 = 2,8571 \times 10^{11}$	$\mu = 0,0$, $\sigma^2 = 2,8571 \times 10^{11}$	$\mu = 0,0$, $\sigma^2 = 2,8571 \times 10^{11}$	Pasa

V-D. Análisis técnico de los casos borde

1. **Arreglo vacío** ($N = 0$): las guardas del controlador y de las rutinas en ensamblador se activan antes de cualquier división o acceso a los punteros. Se retornan estadísticas nulas sin provocar SIGSEGV ni SIGFPE.
2. **Elemento único** ($N = 1$): la varianza es exactamente cero. La comparación con `ucomiss/vucomiss` activa la ruta de copia y conserva el elemento original sin dividir por cero.
3. **Remanente puro** ($N = 7$): como $N < 8$, se obtiene $N \& (\sim 7) = 0$. El bucle AVX2 se omite y todos los elementos se procesan mediante el *tail loop* escalar.
4. **Vector y remanente** ($N = 15$): se procesa un bloque de ocho valores en registros YMM y luego los siete valores restantes de forma escalar, comprobando la transición entre ambas rutas.
5. **Varianza nula** ($N = 1000$): el arreglo constante comprueba que el cálculo produzca $\sigma^2 = 0,0$ y que la guarda impida ejecutar `vdivps` con un divisor nulo.
6. **Valores extremos** ($N = 70$): se mezclan signos y amplitudes desde 10^{-4} hasta 10^6 . El caso verifica que no aparezcan desbordamientos ni subdesbordamientos destructivos en los resultados esperados.

V-E. Dictamen de correctitud

Los seis casos evaluados superaron la validación funcional y la comparación numérica con el oráculo de Python. El mayor error relativo registrado entre las implementaciones fue

$$\text{Err}_{\text{rel,max}} = 1,19 \times 10^{-6} \ll 1,0 \times 10^{-4}. \quad (23)$$

Este resultado se encuentra aproximadamente dos órdenes de magnitud por debajo del umbral establecido y respalda la exactitud numérica de ambas versiones para los casos considerados.

```

Proyecto 1 arqu/proyecto_vectorial/proyecto_vectorial
> make clean && make
rm -rf obj bin
gcc -std=gnu11 -Wall -Wextra -O2 -g -Iinclude -c src/driver.c -o obj/driver.o
nasm -f elf64 -g -F dwarf asm/scalar/stats_scalar.asm -o obj/stats_scalar.o
gcc -std=gnu11 -Wall -Wextra -O2 -g -o bin/norm_scalar obj/driver.o obj/stats_scalar.o -ln
nasm -f elf64 -g -F dwarf asm/vector/stats_vector.asm -o obj/stats_vector.o
gcc -std=gnu11 -Wall -Wextra -O2 -g -o bin/norm_vector obj/driver.o obj/stats_vector.o -ln

Proyecto 1 arqu/proyecto_vectorial/proyecto_vectorial
> python3 tools/run_all_tests.py
[1/3] Compilando con make...
Compilación exitosa.

[2/3] Ejecutando batería de pruebas...
Caso de Prueba | N | Escalar | Vectorial | Max Err Bin | Estado
-----
TC-01: Arreglo vacío (N=0) | 0 | OK | OK | 0,00 | ✓ PASA
TC-02: Elemento único (N=1) | 1 | OK | OK | 0,00e+00 | ✓ PASA
TC-03: Remanente puro (N=7) | 7 | OK | OK | 0,00e+00 | ✓ PASA
TC-04: Vector exacto (N=8) | 8 | OK | OK | 1,19e-07 | ✓ PASA
TC-05: Vector + remanente (N=15) | 15 | OK | OK | 0,00e+00 | ✓ PASA
TC-06: Dos vectores exactos (N=16) | 16 | OK | OK | 5,96e-08 | ✓ PASA
TC-07: Arreglo mediano (N=1000) | 1000 | OK | OK | 1,19e-06 | ✓ PASA
TC-08: Constante / sigma=0 (N=1000) | 1000 | OK | OK | 0,00e+00 | ✓ PASA
TC-09: Valores extremos y negativos | 70 | OK | OK | 2,38e-07 | ✓ PASA

[3/3] RESULTADO FINAL: TODAS LAS PRUEBAS PASARON EXITOSAMENTE (100% CORRECTUD)

```

Figura 1: Ejecución automatizada de la suite de pruebas funcionales.

VI. RESULTADOS DE RENDIMIENTO

VI-A. Metodología de medición

La instrumentación temporal se realizó mediante la función POSIX `clock_gettime(CLOCK_MONOTONIC)`, utilizando marcas de tiempo de alta resolución. La región medida incluye exclusivamente el cómputo de los núcleos `sum_array`, `compute_stats` y `normalize_array`; las operaciones de entrada y salida, como `fread` y `fwrite`, se excluyeron de la ventana de medición.

Para cada tamaño se ejecutaron $R = 30$ repeticiones independientes. Se reportan la media muestral $\bar{T}$ y la desviación estándar muestral s , lo que permite cuantificar la dispersión ocasionada por el sistema operativo, la variación de frecuencia y los efectos térmicos. La ejecución se fijó al núcleo P0 mediante `taskset -c 0`, evitando migraciones entre los P-cores Golden Cove y los E-cores Gracemont del procesador híbrido.

Se seleccionaron cuatro tamaños para recorrer la jerarquía de memoria. La huella indicada considera los arreglos de entrada y salida, ambos en formato `float32`:

- $N = 10^3$: 7,8 KiB, residente en L1d.
- $N = 10^5$: 781,2 KiB, residente en L2.
- $N = 10^6$: 7,63 MiB, residente en L3.
- $N = 2 \times 10^7$: 152,6 MiB, muy superior a la LLC y, por tanto, atendido principalmente desde DRAM.

VI-B. Tiempos de ejecución

La aceleración experimental se define como

$$S = \frac{\bar{T}_{sc}}{\bar{T}_{vec}}, \quad (24)$$

y la eficiencia respecto al ancho ideal de ocho carriles como

$$\eta_{SIMD} \text{ (por ciento)} = \frac{S}{8} \times 100. \quad (25)$$

La Tabla IX presenta los resultados de las 30 repeticiones. Su número de columnas requiere utilizar el ancho completo de la página.

VI-C. Aceleración obtenida

El límite ideal para AVX2 sobre `float32` es $8,0\times$, pues cada registro de 256 bits contiene ocho carriles de 32 bits. La curva de la Figura 2 muestra dos regímenes. Para $N \in [10^3, 10^5]$ se observa una meseta de $5,80\times$ a $5,60\times$; al alcanzar $N = 10^6$, la aceleración disminuye moderadamente a $4,90\times$ dentro de la LLC. Cuando la huella rebasa ampliamente la caché, la curva cae y converge a $2,44\times$ para $N = 2 \times 10^7$, debido al límite de transferencia desde DRAM.

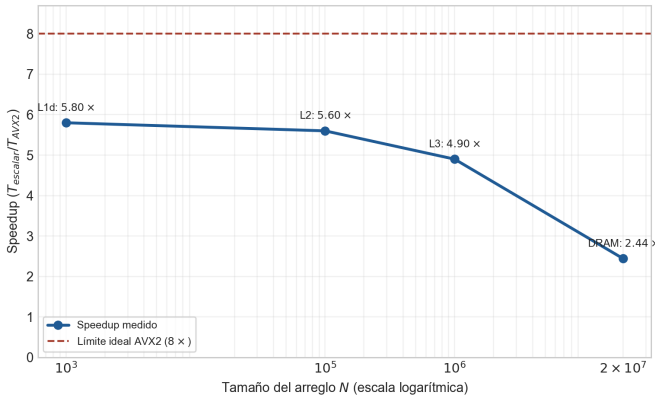


Figura 2: Aceleración de la implementación AVX2 respecto de la versión escalar.

VI-D. Resultados de *perf stat*

Los contadores de la Tabla X se obtuvieron mediante la PMU de Linux con afinidad al núcleo P0. Se comparan un problema residente en L3 ($N = 10^6$) y otro dominado por accesos a DRAM ($N = 2 \times 10^7$).

VI-E. Interpretación de resultados

VI-E1. Régimen limitado por cómputo: Para $N \leq 10^6$, los datos permanecen en L1d, L2 o L3 y la aceleración se mantiene entre $5,80\times$ y $4,90\times$. En $N = 10^6$, AVX2 retira 127.4 millones de instrucciones frente a 916.1 millones de la versión escalar, una reducción de $7,19\times$. El empaquetamiento de ocho valores por instrucción descongiona la etapa de decodificación y el búfer de reordenamiento.

El IPC escalar resulta mayor que el vectorial (2.55 frente a 1.63), pero esta métrica cuenta instrucciones, no operaciones útiles por carril. Una instrucción AVX2 retirada puede realizar

ocho operaciones de punto flotante, por lo que un IPC menor no implica menor rendimiento numérico. El tiempo medido confirma que la versión vectorial completa el trabajo $4,90\times$ más rápido en L3.

La distancia respecto al límite de $8\times$ se explica por el coste de las reducciones horizontales, las conversiones escalares, el control de bucles y los prólogos y epílogos de la ABI. Si se modela la aceleración máxima observada mediante la ley de Amdahl con una fracción paralelizable $p \approx 0,9458$, se obtiene

$$S = \frac{1}{(1-p) + p/8} = \frac{1}{0,0542 + 0,9458/8} \approx 5,80 \times. \quad (26)$$

Este modelo atribuye aproximadamente un 5.42 % del tiempo a trabajo no beneficiado por el ancho vectorial ideal.

VI-E2. Régimen limitado por memoria: Para $N = 2 \times 10^7$, los arreglos de entrada y salida ocupan en conjunto 152,6 MiB, más de quince veces la capacidad de la caché L3 de 10 MiB. La CPU debe solicitar repetidamente líneas de 64 bytes a la memoria principal. Aunque las unidades AVX2 consumen 32 bytes por carga, la tasa de cómputo deja de ser el factor dominante y el rendimiento queda limitado por el ancho de banda y la latencia de DRAM.

La tasa de fallos medida para AVX2 pasa de 9.18 % en L3 a 91.22 % en el caso de DRAM, mientras que su IPC cae de 1.63 a 0.76. Las instrucciones vectoriales continúan reduciendo el número total de instrucciones en aproximadamente $7,17\times$, pero gran parte de los ciclos se consume esperando datos. Esta *Memory Wall* limita el speedup temporal a $2,44\times$, demostrando que el paralelismo aritmético no puede compensar por sí solo una alimentación de datos insuficiente.

VII. EVIDENCIA DE LA SESIÓN DE GDB

VII-A. Propósito técnico y microarquitectónico

La sesión de depuración con GNU GDB auditó en tiempo de ejecución el cumplimiento de la convención System V AMD64 ABI y contrastó el datapath escalar SISD con el datapath SIMD AVX2. En particular, se verificó el procesamiento de un único `float32` en el carril inferior de `xmm3`, frente al procesamiento simultáneo de ocho valores en `yymm0`. También se comprobó la alineación a 32 bytes requerida por `vmovaps` y la igualdad de los resultados almacenados por ambas implementaciones para el caso $N = 16$ (`tc-06_in.dat`).

VII-B. Secuencia de comandos y breakpoints

VII-B1. Kernel escalar: La sesión de `bin/norm_scalar` se configuró con una repetición para aislar el flujo de control. El primer breakpoint se ubicó en la entrada de `normalize_array`, sobre `test edx, edx`, para comprobar los argumentos `rdi=in`, `rsi=out` y `edx=16`. También se inspeccionaron $\mu = -5,637640$ en `xmm0` y $\sigma = 48,620003$ en `xmm1`.

El segundo breakpoint se colocó inmediatamente después de `divss xmm3, xmm1`, dentro de `.norm_loop`. Allí se confirmó que solo el carril inferior de `xmm3` contenía el resultado de la iteración actual y que `eax` avanzaba de uno en uno, equivalente a cuatro bytes por elemento. Finalmente, se eliminó el breakpoint iterativo y se utilizó `finish` antes de inspeccionar la memoria de salida.

Cuadro IX: Tiempo promedio, desviación estándar y aceleración por tamaño ($R = 30$).

N	Nivel jerárquico y huella	Escalar $\bar{T} \pm s$ (ms)	AVX2 $\bar{T} \pm s$ (ms)	S	η_{SIMD}
1,000	L1d (7,8 KiB)	$0,0028 \pm 0,0001$	$0,0005 \pm 0,0000$	$5,80 \times$	72.5 %
100,000	L2 (781,2 KiB)	$0,2943 \pm 0,0042$	$0,0525 \pm 0,0008$	$5,60 \times$	70.0 %
1,000,000	L3 (7,63 MiB)	$2,6637 \pm 0,0381$	$0,5433 \pm 0,0092$	$4,90 \times$	61.3 %
20,000,000	DRAM (152,6 MiB)	$54,5323 \pm 0,8120$	$22,3774 \pm 0,3540$	$2,44 \times$	30.5 %

Cuadro X: Contadores microarquitectónicos obtenidos mediante `perf stat` en el núcleo P0 Golden Cove.

Métrica	$N = 10^6$ L3, escalar	$N = 10^6$ L3, AVX2	$N = 2 \times 10^7$ DRAM, escalar	$N = 2 \times 10^7$ DRAM, AVX2
Ciclos	359,663,277	78,283,929	7,410,341,145	3,356,712,357
Instrucciones	916,136,472	127,443,561	18,313,758,611	2,552,451,534
IPC	2.55	1.63	2.47	0.76
Referencias de caché	10,091,746	9,981,784	200,364,360	199,071,098
Fallos de caché	1,918,819	915,922	140,654,647	181,600,111
Tasa de fallos	19.01 %	9.18 %	70.20 %	91.22 %

```

1 (gdb) file bin/norm_scalar
2 (gdb) set args data/test_suite/tc-06_in.dat data/
   test_suite/tc-06_sc_out.dat 1
3 (gdb) break asm/scalar/stats_scalar.asm:167
4 (gdb) run
5 (gdb) print $rdi
6 (gdb) print $rsi
7 (gdb) print $edx
8 $edx = 16
9 (gdb) print $xmm0.v4_float
10 # carril 0: mean = -5.637640
11 (gdb) print $xmm1.v4_float
12 # carril 0: stddev = 48.620003
13 (gdb) break asm/scalar/stats_scalar.asm:184
14 (gdb) continue
15 (gdb) print $eax
16 $eax = 0
17 (gdb) print $xmm3.v4_float
18 # un resultado activo en el carril 0
19 (gdb) delete 2
20 (gdb) finish
21 (gdb) print (void*)out
22 (gdb) x/8fw out

```

Listing 9: Comandos principales de la sesión GDB escalar.

```

$ gdb -q bin/norm_scalar
Reading symbols from bin/norm_scalar...
(gdb) set args data/test_suite/tc-06_in.dat data/test_suite/tc-06_sc_out.dat 1
(gdb) break asm/scalar/stats_scalar.asm:167
Punto de interrupción 1 at 0x55555555a200: file asm/scalar/stats_scalar.asm, line 167.
(gdb) run
Starting program: /home/lain/Escritorio/Proyecto 1 arqui/proyecto_vectorial/proyecto_vectorial/bin/norm_scalar data/test_suite/tc-06_in.dat data/test_suite/tc-06_sc_out.dat 1

This GDB supports auto-downloading debuginfo from the following URLs:
 * https://debuginfod.ubuntu.com/
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[Separación de hilo usando libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".

Breakpoint 1, normalize_array () at asm/scalar/stats_scalar.asm:167
167 test = out; out
(gdb) info registers rdi rdx
rdi    0x55555555a200    93824992256512
rdx    0x555555559520    93824992253216
(gdb) print $xmm0.v4_float
$ = {-5.63764095, 0, 0, 0}
(gdb) print $xmm1.v4_float
$ = {48.6200027, 0, 0, 0}
(gdb)

```

Figura 3: Recepción de argumentos y parámetros flotantes en el kernel escalar.

```

(gdb) break asm/scalar/stats_scalar.asm:184
Punto de interrupción 2 at 0x55555555a2b: file asm/scalar/stats_scalar.asm, line 184.
(gdb) continue
Continuando.

Breakpoint 2, normalize_array.norm_loop () at asm/scalar/stats_scalar.asm:184
184 movss    rsi, rax + 4], xmm3; out[i] = xmm3
(gdb) print $eax
$3 = 0
(gdb) print $xmm3.v4_float[0]
$4 = 0.899751365
(gdb)

```

Figura 4: Procesamiento SISD de un elemento float32.

```

(gdb) finish
Correr hasta la salida desde #0 normalize_array.norm_loop () at asm/scalar/stats_scalar.asm:184
main (argc=<optimized out>, argv=<optimized out>) at src/driver.c:140
140 clock_gettime(CLOCK_MONOTONIC, &t1);
(gdb) print (void*)out
$3 = (void *) 0x555555559520
(gdb) x/8fw out
0x555555559520: 0.899751365    1.82348073    -0.0602805689    -0.851318777
0x555555559530: -1.11406946    -1.66397166    0.220615491     0.60411936
(gdb)

```

Figura 5: Memoria de salida producida por la versión escalar.

VII-B2. Kernel vectorial: La sesión de `bin/norm_vector` utilizó el mismo arreglo y una única repetición. El breakpoint de entrada, situado sobre `test edx`, `edx`, permitió validar los argumentos antes de que `vbroadcastss` replicara μ y σ en `ymm4` y `ymm5`. El segundo breakpoint se ubicó después de `vdivps ymm0, ymm0, ymm5`; en ese punto, `ymm0` contenía ocho resultados normalizados y `eax` avanzaba en bloques de ocho elementos, equivalentes a 32 bytes.

La expresión `((unsigned long)out) % 32` produjo cero, confirmando que el búfer de salida satisface la alineación exigida por las cargas y almacenamientos con `vmovaps`. Tras eliminar el breakpoint del bucle, `finish` permitió inspeccionar el mismo intervalo de memoria que en la versión escalar.

```

1 (gdb) file bin/norm_vector
2 (gdb) set args data/test_suite/tc-06_in.dat data/
   test_suite/tc-06_vc_out.dat 1
3 (gdb) break asm/vector/stats_vector.asm:248
4 (gdb) run
5 (gdb) print $edx
6 $edx = 16
7 (gdb) print $xmm0.v4_float
8 # mean = -5.637640
9 (gdb) print $xmm1.v4_float
10 # stddev = 48.620003
11 (gdb) break asm/vector/stats_vector.asm:271
12 (gdb) continue

```

```

13 (gdb) print $eax
14 $eax = 0
15 (gdb) print $ymm0.v8_float
16 # ocho resultados normalizados simultaneos
17 (gdb) print ((unsigned long)out) % 32
18 $ = 0
19 (gdb) delete 2
20 (gdb) finish
21 (gdb) x/8fw out

```

Listing 10: Comandos principales de la sesión GDB vectorial.

```

$ gdb -q bin/norm_vector
Reading symbols from bin/norm_vector...
(gdb) set args data/test_suite/tc-06_in.dat data/test_suite/tc-06_vc_out.dat 1
(gdb) break asm/vector/stats_vector.asm:248
Punto de interrupción 1 at 0x1001: file asm/vector/stats_vector.asm, line 248.
(gdb) run
Starting program: /home/lain/Escritorio/Proyecto 1 arqui/proyecto_vectorial/proyecto_vectorial/bin/norm_vector data/test_suite/tc-06_in.dat data/test_suite/tc-06_vc_out.dat 1
This GDB supports auto-downloading debuginfo from the following URLs:
<https://debuginfod.ubuntu.com>
Enable debuginfod for this session? (y or [n]) y
Debuginfod has been enabled.
To make this setting permanent, add 'set debuginfod enabled on' to .gdbinit.
[depración de hilo usando libthread_db enabled]
Using host libthread_db library "/usr/lib/x86_64-linux-gnu/libthread_db.so.1".
Breakpoint 1, normalize_array () at asm/vector/stats_vector.asm:248
248      test     %eax, %eax
(gdb) print $ymm0.v8_float
$1 = {-5.63764048, 0, 0, 0, 0, 0, 0, 0}
(gdb) print $ymm1.v8_float
$2 = {48.6200027, 0, 0, 0, 0, 0, 0, 0}
(gdb)

```

Figura 6: Argumentos de entrada de la rutina vectorial.

```

(gdb) break asm/vector/stats_vector.asm:271
Punto de interrupción 2 at 0x555555555b37: file asm/vector/stats_vector.asm, line 271.
(gdb) continue
Continuando.
Breakpoint 2, normalize_array_norm_vec_loop () at asm/vector/stats_vector.asm:271
271      vmovaps    rsi, rax*4, ymm0 ; guarda 8 floats alineados
(gdb) print $ymm0.v8_float
$3 = {0.899751365, 1.82348073, -0.0602805801, -0.851318777, -1.11406946, -1.66397166, 0.220615491, 0.60411936}
(gdb)

```

Figura 7: Procesamiento SIMD de ocho elementos float32.

```

(gdb) finish
Correr hasta la salida desde #0 normalize_array () at asm/vector/stats_vector.asm:248
main (argc=<optimized out>, argv=<optimized out>) at src/driver.c:140
140      clock_gettime(CLOCK_MONOTONIC, &ti);
(gdb) print (void*)out
$4 = (void*) 0x555555555920
(gdb) print ((unsigned long)out) % 32
$5 = 0
(gdb) x/8fw out
0x555555555920: 0.899751365 1.82348073 -0.0602805801 -0.851318777
0x555555555930: -1.11406946 -1.66397166 0.220615491 0.60411936
(gdb)

```

Figura 8: Alineación y memoria de salida de la versión AVX2.

VII-C. Comparación de hallazgos en bajo nivel

La Tabla XI conserva el ancho de una columna y resume las diferencias observadas directamente en GDB.

Cuadro XI: Hallazgos de la inspección escalar y vectorial.

Dimensión	Escalar	AVX2
Paradigma	SISD, SSE escalar	SIMD empaquetado
Registro	ymm3, carril 0	ymm0, ocho carriles
Elementos/iteración	1	8
Paso del índice	+1 elemento (+4 B)	+8 elementos (+32 B)
Aritmética	subss, divss	vsubps, vdivps
Acceso	movss, 32 bits	vmovaps, 256 bits alineados
Parámetros	μ y σ en XMM	Broadcast a ymm4 y ymm5
Iteraciones ($N = 16$)	16	2
Alineación	4 bytes suficiente	32 bytes en esta implementación
Salida inicial	[0,899751, 1,823481, ...]	[0,899751, 1,823481, ...]
Epílogo	Sin limpieza AVX	vzeroupper

La observación confirma una reducción del 87.5 % en el número de iteraciones para $N = 16$. AVX2 ocupa los ocho

carriles de un registro YMM y reduce la sobrecarga de control, mientras que la versión escalar utiliza únicamente un valor de 32 bits por instrucción. La alineación a 32 bytes evita accesos desalineados con vmovaps; vzeroupper, por su parte, prepara el retorno a código que pueda utilizar SSE heredado.

VII-D. Convergencia numérica observada

La inspección x/8fw out produjo en ambos ejecutables el mismo bloque inicial:

$$\text{out}[0 \dots 7] = [0,899751365, 1,823480730, -0,0602805801, -0,851318777, -1,114069460, -1,663971660, 0,220615491, 0,604119360]. \quad (27)$$

Para estos ocho elementos, la diferencia máxima observada fue

$$\max_{0 \leq i < 8} |\text{out}_{\text{sc}}[i] - \text{out}_{\text{vec}}[i]| = 0,0, \quad (28)$$

valor inferior a la epsilon de máquina de float32, $\epsilon \approx 1,19 \times 10^{-7}$. La evidencia confirma la coincidencia exacta del bloque inspeccionado, además de la equivalencia numérica global establecida por la suite de pruebas.

VIII. CONCLUSIONES, LIMITACIONES Y TRABAJO FUTURO

VIII-A. Conclusiones

La vectorización mediante AVX2 de 256 bits demostró la eficacia del paralelismo a nivel de datos para el cálculo de estadísticos y la normalización de arreglos float32. La implementación alcanzó un speedup máximo de 5,80×, equivalente al 72.5 % del límite ideal de 8,0×, y redujo en 7,19× el número de instrucciones retiradas en el caso residente en L3. Esta mejora se obtuvo sin comprometer la exactitud: el error relativo máximo de la suite fue $1,19 \times 10^{-6}$, muy inferior a la tolerancia establecida de $1,0 \times 10^{-4}$.

Los resultados también evidencian la transición entre dos regímenes de ejecución. Mientras los datos permanecen en la jerarquía de cachés, para $N \leq 10^6$, la aplicación se encuentra principalmente limitada por cómputo y alcanza aceleraciones entre 5,80× y 4,90×. Al utilizar $N = 2 \times 10^7$, cuya huella combinada es de 152,6 MiB, el problema pasa a estar limitado por memoria. La tasa de fallos de caché de la versión AVX2 alcanza 91.22 % y el speedup desciende a 2,44×, confirmando el efecto de la *Memory Wall*.

La ley de Amdahl explica la distancia restante respecto al factor ideal de ocho. Las reducciones horizontales con vextractf128 y vhaddps, las conversiones escalares, el control de los bucles y el procesamiento del remanente cuando $N \bmod 8 \neq 0$ constituyen una fracción que no obtiene el beneficio completo del ancho SIMD. Este coste fijo tiene un peso relativo mayor en arreglos pequeños.

Finalmente, la implementación mostró robustez en el nivel de arquitectura y de interfaz binaria. El cumplimiento de System V AMD64 ABI permitió integrar los núcleos NASM con el controlador en C; la asignación alineada a 32 bytes

evitó accesos inválidos con `vmovaps`; y `vzeroupper` dejó limpios los carriles superiores antes de retornar a código potencialmente basado en SSE. La inspección con GDB confirmó tanto la alineación como la equivalencia de las salidas escalares y vectoriales.

VIII-B. Limitaciones observadas

La limitación dominante para arreglos que exceden la LLC es el ancho de banda de DRAM. En este régimen, la capacidad de las unidades vectoriales para consumir y procesar datos supera la tasa con la que la memoria principal puede entregar líneas de caché. En consecuencia, las unidades SIMD permanecen parcialmente inactivas mientras el pipeline espera datos, por lo que aumentar únicamente el ancho vectorial no produce una aceleración proporcional.

La implementación también es estrictamente monohilo y utiliza un solo núcleo físico. Por ello, no aprovecha el paralelismo a nivel de hilos ni los restantes P-cores y E-cores del procesador. Los resultados caracterizan el beneficio de SIMD dentro de un núcleo, pero no representan el rendimiento máximo agregado de la plataforma completa.

Adicionalmente, las sumas se realizan en precisión simple y su orden cambia entre la versión escalar y la vectorial. Aunque las diferencias permanecieron dentro de la tolerancia definida, aplicaciones con requisitos numéricos más estrictos podrían necesitar acumuladores de doble precisión o algoritmos de suma compensada.

VIII-C. Trabajo futuro

Una primera extensión consiste en emplear FMA3 durante la acumulación de la varianza. Después de calcular $d_i = x_i - \mu$, una instrucción como `vfmadd231ps` puede evaluar y acumular d_i^2 en una única operación fusionada. Esto reduce el número de instrucciones y evita el redondeo intermedio separado entre la multiplicación y la suma.

También se propone desenrollar los bucles vectoriales por factores de dos o cuatro y mantener varios acumuladores YMM independientes. Esta organización reduce las dependencias RAW de las cadenas de suma y ofrece al planificador fuera de orden más operaciones independientes para utilizar los puertos de ejecución disponibles.

Para arreglos mucho mayores que la LLC, podría evaluarse el uso de almacenamientos no temporales mediante `vmovntps`. Si la salida no será reutilizada inmediatamente, estas escrituras pueden disminuir el tráfico de *write-allocate* y reducir la contaminación de las cachés. Su beneficio debe medirse experimentalmente, pues depende del tamaño, la alineación y el patrón de reutilización.

Por último, una implementación híbrida SIMD+OpenMP permitiría dividir el arreglo entre varios hilos y combinar el paralelismo AVX2 dentro de cada núcleo con paralelismo a nivel de hilos. Esta alternativa deberá considerar la topología híbrida del procesador, la afinidad de los hilos y la saturación compartida del ancho de banda de memoria, que probablemente se convierta nuevamente en el límite para tamaños grandes.

APÉNDICE A DIAGRAMAS DE ARQUITECTURA

La Figura 9 resume el flujo jerárquico del sistema, desde la orquestación en C hasta la ejecución de los kernels escalares y vectoriales en los registros del procesador.

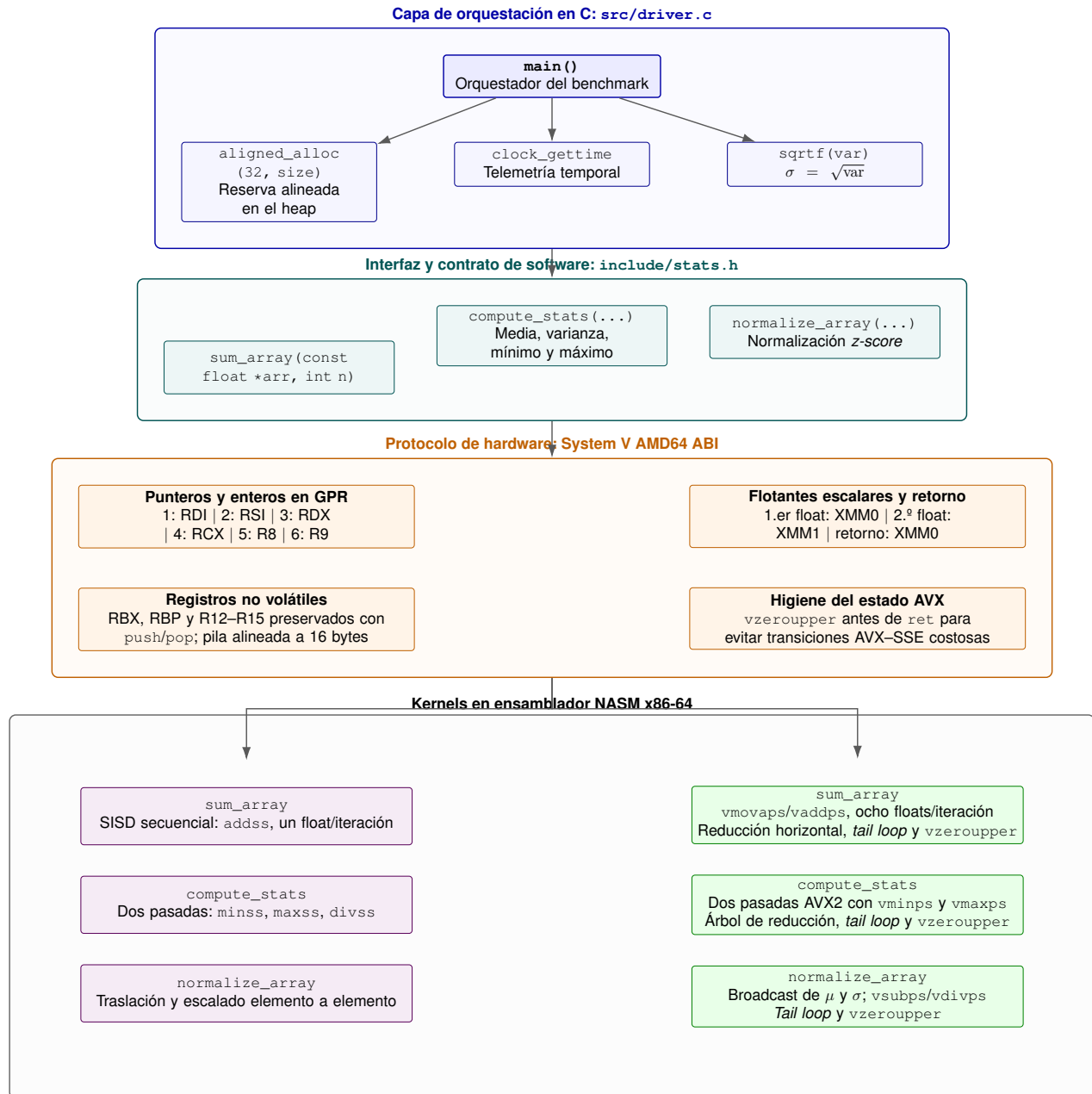


Figura 9: Flujo jerárquico desde la capa de orquestación en C hasta los backends escalar y AVX2 ejecutados por el hardware.

A-A. Bucle escalar SISD paso a paso

El kernel escalar recorre el arreglo de manera estrictamente secuencial. Cada iteración carga un único valor `float32`, lo acumula en el carril inferior de `xmm0` y avanza el índice una posición. Al no existir particionamiento vectorial, la rutina no necesita calcular un límite por bloques ni realizar una reducción horizontal al finalizar.

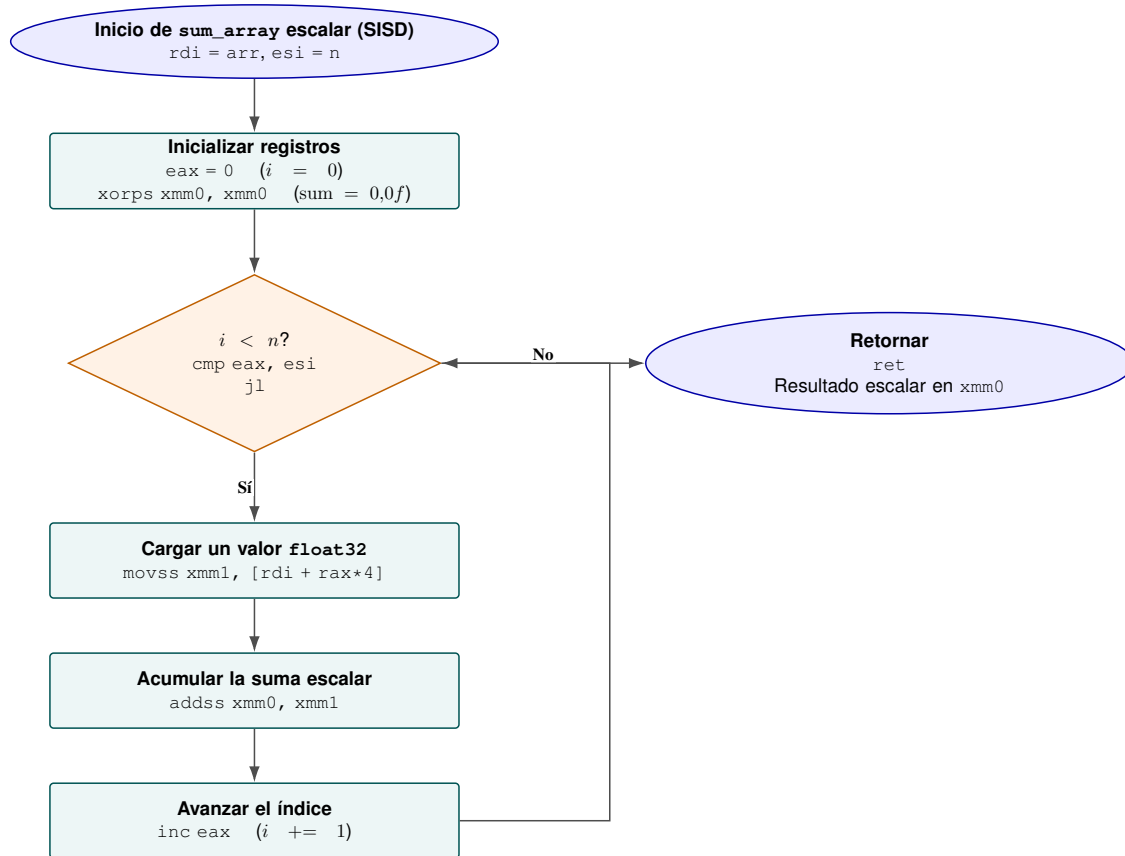


Figura 10: Flujo de ejecución del bucle escalar SISD de `sum_array`.

A-B. Bucle vectorial AVX2 con remanente

El kernel vectorial procesa ocho valores `float32` por iteración. Antes de entrar al bucle calcula $N_{\text{vec}} = n \& (\sim 7)$, reduce horizontalmente los acumuladores al concluir los bloques completos y procesa los $n \bmod 8$ elementos restantes mediante un bucle escalar de cierre.

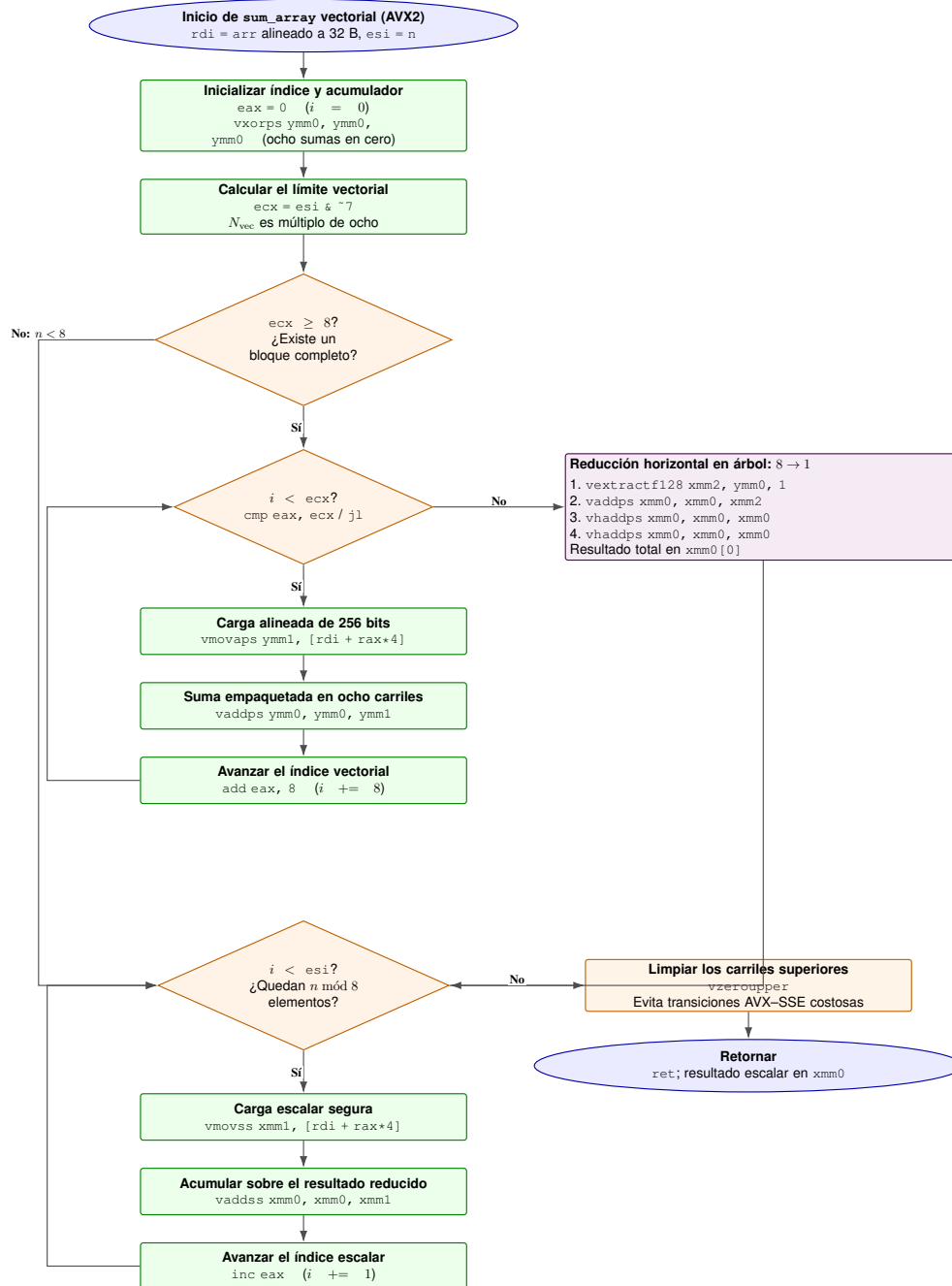


Figura 11: Flujo del bucle AVX2, la reducción horizontal y el procesamiento escalar del remanente.

A-C. Normalización vectorial con guarda de desviación nula

La rutina `normalize_array` calcula

$$\text{out}[i] = \frac{\text{in}[i] - \mu}{\sigma}. \quad (29)$$

Los parámetros escalares μ y σ se replican en los ocho carriles de registros YMM mediante `vbroadcastss`. Antes de efectuar cualquier división, la rutina comprueba si $\sigma = 0,0$; en ese caso ejecuta una copia directa para evitar valores NaN o infinitos. La Figura 12 presenta ambas rutas y sus bucles escalares de remanente.

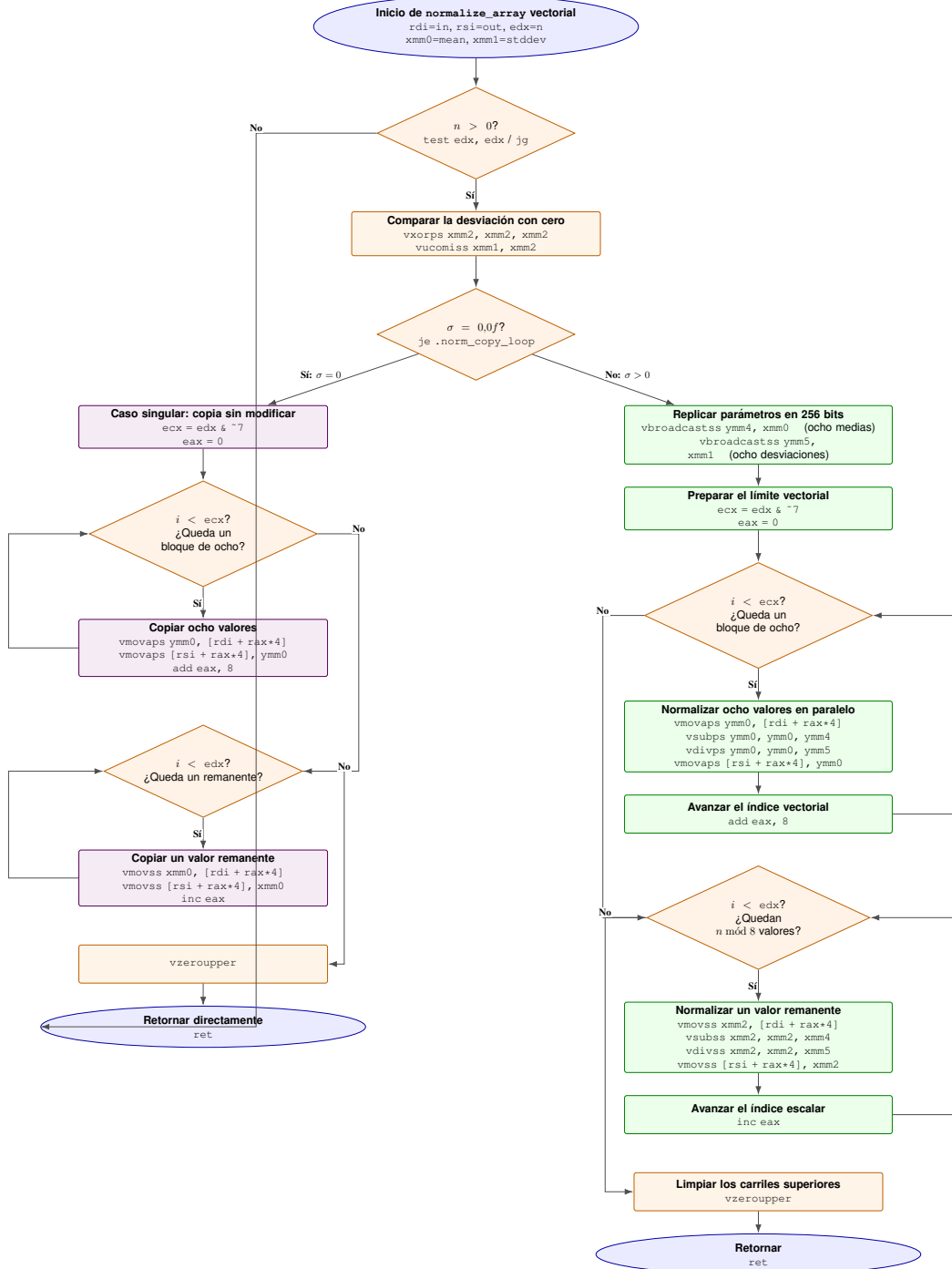


Figura 12: Flujo de `normalize_array`: guarda para $\sigma = 0$, copia directa y normalización vectorial con bucles de remanente.

A-D. Flujo y validación de memoria

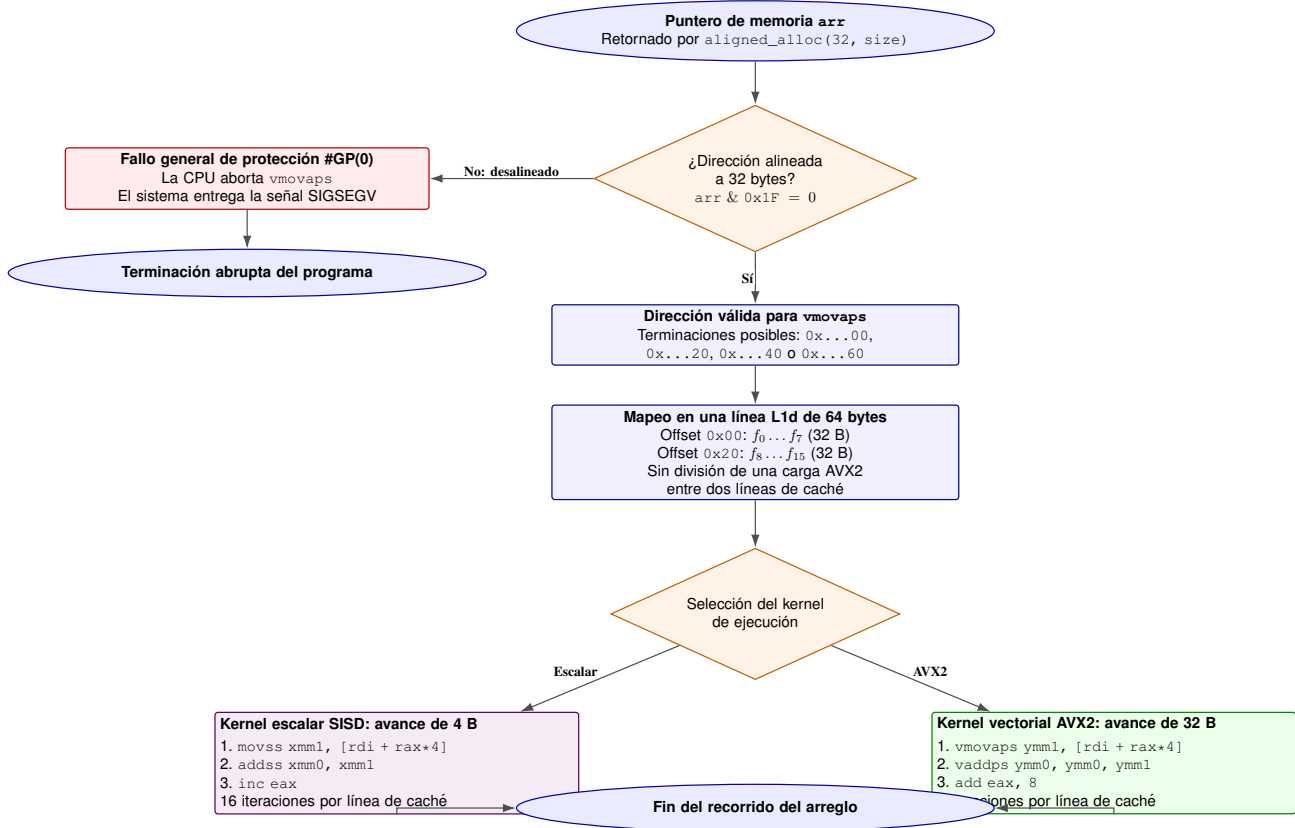


Figura 13: Validación de alineación, mapeo en líneas de caché y patrones de recorrido escalar y AVX2.

A-E. Diagrama 3: Tablas exhaustivas de asignación de registros

Cuadro XII: Convención de registros en System V AMD64 ABI.

Clase ABI	Registros de hardware	Regla de preservación	Comportamiento en subrutinas
Callee-saved (no volátiles)	RBX, RBP, R12–R15, RSP	Obligatorio preservar	Si la función los modifica, debe guardarlos en la pila y restaurarlos antes de retornar.
Caller-saved (volátiles)	RAX, RCX, RDX, RSI, RDI, R8–R11	Libre modificación	La función llamada puede utilizarlos sin preservación; el invocador asume que se destruyen tras un <code>call</code> .
SIMD volátiles	XMM0–XMM15 / YMM0–YMM15	Libre modificación	Todos los registros vectoriales son volátiles bajo la ABI System V AMD64.

Cuadro XIII: Asignación de registros de `sum_array`: implementación escalar.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RDI	GPR 64 bits	Caller-saved	Entrada/bucle	Puntero base al arreglo de entrada <code>arr</code> .
ESI	GPR 32 bits	Caller-saved	Entrada/bucle	Límite superior n y condición de parada.
EAX/RAX	GPR 32/64 bits	Caller-saved	Bucle escalar	Índice i ; direccionamiento <code>[rdi + rax*4]</code> .
XMM0	SSE 128 bits	Caller-saved	Bucle/retorno	Acumulador escalar inicializado con <code>xorps</code> ; valor de retorno en el carril 0.
XMM1	SSE 128 bits	Caller-saved	Bucle escalar	Carga temporal del elemento <code>arr[i]</code> mediante <code>movss</code> .

Cuadro XIV: Asignación de registros de `sum_array`: implementación AVX2.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RDI	GPR 64 bits	Caller-saved	Entrada/bucles	Puntero base <code>arr</code> , alineado a 32 bytes.
ESI	GPR 32 bits	Caller-saved	Entrada/remanente	Cantidad total de elementos n .
ECX	GPR 32 bits	Caller-saved	Particionado/AVX2	Límite $N_{vec} = n \& (\sim 7)$.
EAX/RAX	GPR 32/64 bits	Caller-saved	AVX2/remanente	Índice i ; avanza ocho elementos en AVX2 y uno en el remanente.
YMM0	AVX 256 bits	Caller-saved	Bucle AVX2	Acumulador de ocho sumas parciales, inicializado con <code>vxorps</code> .
YMM1	AVX 256 bits	Caller-saved	Bucle AVX2	Carga alineada de ocho valores mediante <code>vmovaps</code> .
XMM2	SSE 128 bits	Caller-saved	Reducción	Mitad alta de YMM0, extraída con <code>vextractf128</code> .
XMM0	SSE 128 bits	Caller-saved	Reducción/remanente/retorno	Acumulador colapsado con <code>vhaddps</code> ; recibe el remanente y retorna el resultado en el carril 0.
XMM1	SSE 128 bits	Caller-saved	Remanente escalar	Carga del valor restante mediante <code>vmovss</code> .

Cuadro XV: Asignación de registros de `compute_stats`: implementación escalar.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
R12	GPR 64 bits	Callee-saved	Pasadas 1 y 2	Puntero base <code>arr</code> , recibido inicialmente en RDI.
R13D	GPR 32 bits	Callee-saved	Pasadas 1 y 2	Tamaño n , recibido inicialmente en ESI.
R14	GPR 64 bits	Callee-saved	Fin de pasada 1	Puntero <code>mean*</code> , recibido en RDX.
R15	GPR 64 bits	Callee-saved	Fin de pasada 2	Puntero <code>var*</code> , recibido en RCX.
RBX	GPR 64 bits	Callee-saved	Fin de pasada 1	Puntero <code>min*</code> , recibido en R8.
RBP	GPR 64 bits	Callee-saved	Fin de pasada 1	Puntero <code>max*</code> , recibido en R9.
EAX/RAX	GPR 32/64 bits	Caller-saved	Pasadas 1 y 2	Índice i : $1 \dots n - 1$ en P1 y $0 \dots n - 1$ en P2.
XMM0	SSE 128 bits	Caller-saved	Pasada 1	Acumulador de $\sum x_i$.
XMM1	SSE 128 bits	Caller-saved	Pasada 1	Acumulador del mínimo mediante <code>minss</code> .
XMM2	SSE 128 bits	Caller-saved	Pasada 1	Acumulador del máximo mediante <code>maxss</code> .
XMM3	SSE 128 bits	Caller-saved	Pasadas 1 y 2	Registro temporal para <code>arr[i]</code> y la desviación respecto a la media.
XMM4	SSE 128 bits	Caller-saved	Pasadas 1 y 2	Conversión de n a punto flotante mediante <code>cvtsi2ss</code> .
XMM5	SSE 128 bits	Caller-saved	Fin P1/P2	Media $\mu = \text{sum}/n$, conservada para la segunda pasada.
XMM6	SSE 128 bits	Caller-saved	Pasada 2	Acumulador de $\sum (x_i - \mu)^2$.

Cuadro XVI: Asignación de registros de `compute_stats`: implementación AVX2.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
R12	GPR 64 bits	Callee-saved	Pasadas 1 y 2	Puntero base <code>arr</code> , alineado a 32 bytes.
R13D	GPR 32 bits	Callee-saved	Pasadas 1 y 2	Tamaño total n .
R14	GPR 64 bits	Callee-saved	Fin de pasada 1	Destino <code>mean*</code> .
R15	GPR 64 bits	Callee-saved	Fin de pasada 2	Destino <code>var*</code> .

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RBX	GPR 64 bits	Callee-saved	Fin de pasada 1	Destino $\min^*$.
RBP	GPR 64 bits	Callee-saved	Fin de pasada 1	Destino $\max^*$.
ECX	GPR 32 bits	Callee-saved	Particionado P1/P2	Límite $N_{\text{vec}} = n \ \& \ (\sim 7)$.
EAX/RAX	GPR 32/64 bits	Callee-saved	Índice P1/P2	Avanza ocho elementos en AVX2 y uno en el remanente.
YMM0	AVX 256 bits	Callee-saved	P1/P2 vectoriales	P1: acumulador de suma; P2: acumulador de desviaciones cuadráticas.
YMM1	AVX 256 bits	Callee-saved	P1/P2 vectoriales	P1: acumulador mínimo; P2: carga y cálculo de $(x - \mu)^2$.
YMM2	AVX 256 bits	Callee-saved	P1 vectorial	Acumulador máximo mediante <code>vmaxps</code> .
YMM3	AVX 256 bits	Callee-saved	P1/P2 vectoriales	P1: carga de ocho valores; P2: difusión de la media.
XMM0	SSE 128 bits	Callee-saved	Reducciones P1/P2	Reducción de suma y de varianza.
XMM1	SSE 128 bits	Callee-saved	Reducción/remanente	Mínimo colapsado en P1 y temporal de diferencias en el remanente de P2.
XMM2	SSE 128 bits	Callee-saved	Reducción/remanente	Máximo colapsado y auxiliar escalar.
XMM3	SSE 128 bits	Callee-saved	Reducción/remanente	Auxiliar para <code>vshufps</code> y operaciones del remanente.
XMM4	SSE 128 bits	Callee-saved	Divisor P1/P2	Representación flotante de n mediante <code>vcvtsi2ss</code> .
XMM5	SSE 128 bits	Callee-saved	Pasadas 1 y 2	Media escalar μ , utilizada en la difusión y el remanente.

Cuadro XVII: Asignación de registros de `normalize_array`: implementación escalar.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RDI	GPR 64 bits	Callee-saved	Entrada/bucle	Puntero al arreglo de entrada <code>in</code> .
RSI	GPR 64 bits	Callee-saved	Entrada/bucle	Puntero al arreglo de salida <code>out</code> .
EDX	GPR 32 bits	Callee-saved	Entrada/bucle	Tamaño n .
XMM0	SSE 128 bits	Callee-saved	Entrada/bucle	Media escalar μ .
XMM1	SSE 128 bits	Callee-saved	Entrada/bucle	Desviación estándar σ .
XMM2	SSE 128 bits	Callee-saved	Detección de borde	Cero escalar para <code>ucomiss xmm1, xmm2</code> .
EAX/RAX	GPR 32/64 bits	Callee-saved	Bucle	Índice $i = 0 \dots n - 1$.
XMM3	SSE 128 bits	Callee-saved	Bucle	Temporal que contiene <code>in[i]</code> , <code>in[i] - μ</code> y el resultado normalizado.

Cuadro XVIII: Asignación de registros de `normalize_array`: implementación AVX2.

Registro	Tipo y ancho	Estado ABI	Fase	Rol técnico y semántica
RDI	GPR 64 bits	Callee-saved	Entrada/bucles	Puntero base <code>in</code> , alineado a 32 bytes.
RSI	GPR 64 bits	Callee-saved	Entrada/bucles	Puntero base <code>out</code> , alineado a 32 bytes.
EDX	GPR 32 bits	Callee-saved	Entrada/bucles	Tamaño n .
XMM0	SSE 128 bits	Callee-saved	Entrada/difusión	Media escalar μ recibida desde <code>C</code> .
XMM1	SSE 128 bits	Callee-saved	Entrada/difusión	Desviación estándar σ recibida desde <code>C</code> .
XMM2	SSE 128 bits	Callee-saved	Detección/remanente	Comparación $\sigma = 0,0$ y temporal escalar.
YMM4	AVX 256 bits	Callee-saved	AVX2/remanente	Broadcast $[\mu, \mu, \mu, \mu, \mu, \mu, \mu, \mu]$.
YMM5	AVX 256 bits	Callee-saved	AVX2/remanente	Broadcast $[\sigma, \sigma, \sigma, \sigma, \sigma, \sigma, \sigma, \sigma]$.
ECX	GPR 32 bits	Callee-saved	Particionado AVX2	Límite $N_{\text{vec}} = n \ \& \ (\sim 7)$.
EAX/RAX	GPR 32/64 bits	Callee-saved	AVX2/remanente	Índice i ; paso de ocho en vector y uno en el remanente.
YMM0	AVX 256 bits	Callee-saved	Bucle AVX2	Carga de ocho valores, resta, división y almacenamiento empaquetados.